Screenshots — Terraform Backend

Lecture 8 · 32 shots, chronological · source stills from the session (2026-10-05 23:21 – 2026-10-06 00:21)

Terraform Backend

A backend defines where Terraform stores its state data files.

The primary function of a backend is to store the state created by Terraform runs after provisioning our resources. This gives Terraform a single place to look up what the expectation of our resources is from the last time we applied.

How to save terraform state file?

1. Saving Locally -

A local backend stores the state file on the machine where Terraform is running. This is the default backend that is used if you don't specify a backend in your Terraform configuration. **The local backend is useful for testing and development environments, but it's not recommended for production environments** since it can lead to inconsistencies if the state file is lost or corrupted.

2. Saving Remotely - A remote backend stores the state file in a centralized location, such as a cloud object storage service or a database. **Remote backends provide several benefits, such as enabling collaboration between team members, versioning state files, and providing a history of changes.** There are several remote backend providers available, **such as Amazon S3, Azure Storage, Google Cloud Storage etc.**

Advantages of using Remote Backend

Using a remote backend has several advantages, such as:

1. Centralized storage of the state file, which enables collaboration between team members working on the same project.
2. **Versioning of state files**, which allows you to manage and track changes to your infrastructure over time.
3. History of changes, which can help with troubleshooting and auditing.
4. Features like remote **state locking** prevent conflicts and ensure consistency.

What is locking and why do we need it?

If the state file is stored remotely so that many people can access it, then you risk multiple **people attempting to make changes to the same file at the exact same time**. Simultaneous operations could lead to conflicting changes**, potentially corrupting the state**.

So we need to provide a mechanism that will "lock" the state if its currently in-use by another user. The feature of Terraform state locking is supported by AWS S3 and Dynamo DB.

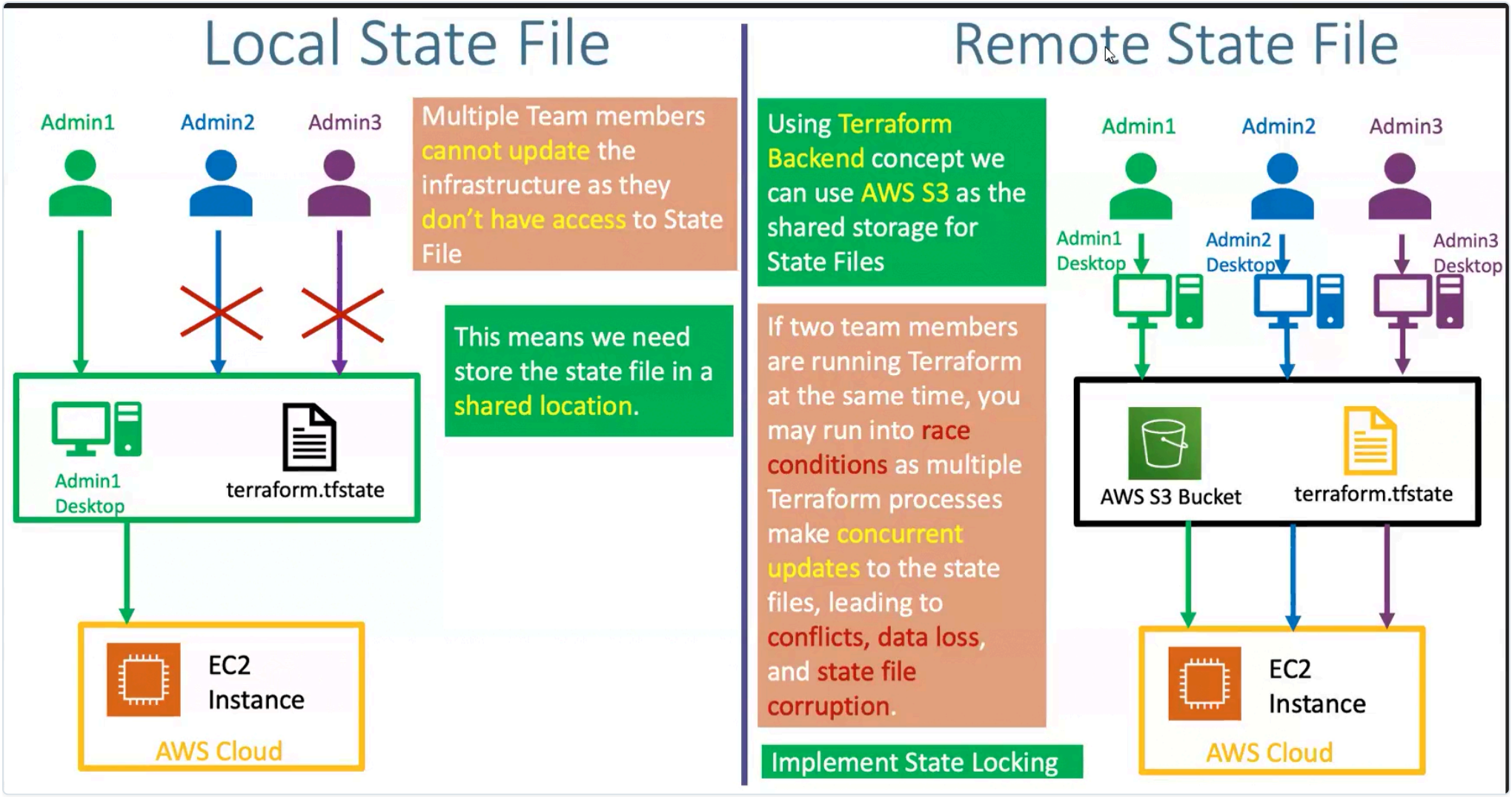
Terraform Backend Initialization

Terraform backend should be configured like any other configuration in the configuration file and when you run the terraform init, Backed will be created.

For example, we are going to configure the AWS S3 as a Terraform backend with dynamodb.

This is the code that creates the backend. But is it so easy?

```
terraform {
  backend "s3" {
    bucket         = "angelo-terraform-state-backend"
    key            = "terraform.tfstate"
    region         = "eu-west-1"
    dynamodb_table = "terraform_state"
  }
}
```


Local State File
Admin1
Admin2
Admin3
Multiple Team members cannot update the infrastructure as they don't have access to State File
This means we need store the state file in a shared location.
Admin1 Desktop
terraform.tfstate
EC2 Instance
AWS Cloud
Remote State File
Using Terraform Backend concept we can use AWS S3 as the shared storage for State Files
If two team members are running Terraform at the same time, you may run into race conditions as multiple Terraform processes make concurrent updates to the state files, leading to conflicts, data loss, and state file corruption.
Implement State Locking
Admin1
Admin2
Admin3
Admin1 Desktop
Admin2 Desktop
Admin3 Desktop
AWS S3 Bucket
terraform.tfstate
EC2 Instance
AWS Cloud


Instances | EC2 | ap-south-1
S3 buckets | S3 | ap-south-1
DynamoDB | ap-south-1

Terraform backend store...

Recommended Production Setup>>>>>

A common production architecture:

Terraform

↓

S3 Bucket (state storage)

↓

DynamoDB Table (state locking)

Benefits:

- Safe collaboration
- Prevents simultaneous updates
- Versioning support

Why Locking Matters

State locking prevents situations where:

·Two engineers run terraform apply simultaneously

·One process overwrites another's state

·Infrastructure drift or corruption occurs

```
resource "aws_s3_bucket" "example" {
  bucket = "ramprateek"
 force_destroy = true
  tags = {
        Name = "Terraform State File Storage"
}
}

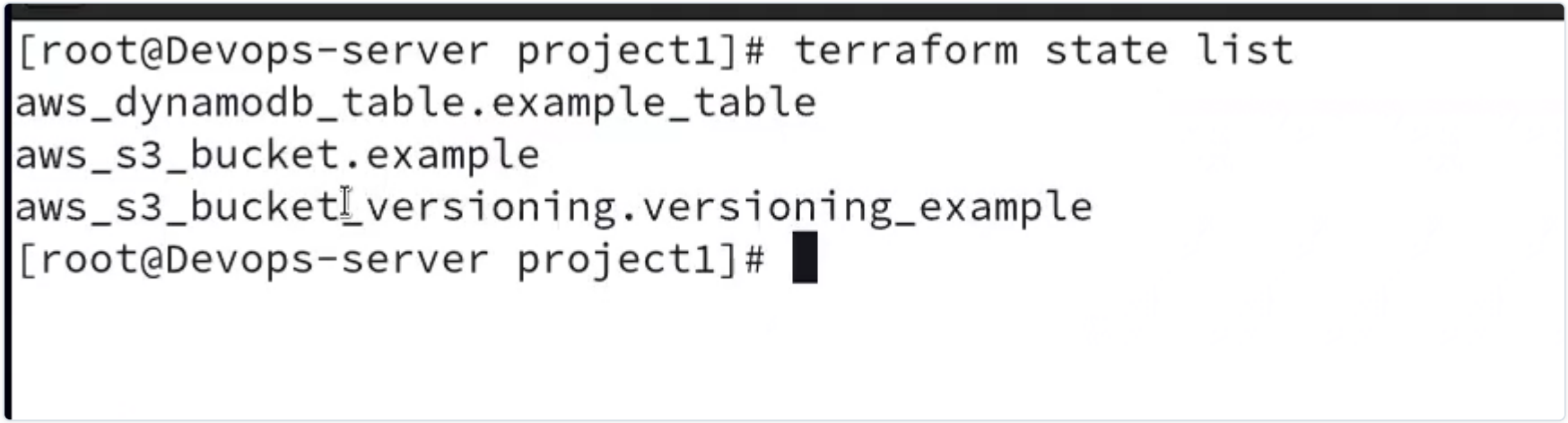
resource "aws_s3_bucket_versioning" "versioning_example" {
  bucket = aws_s3_bucket.example.id
  versioning_configuration {
    status = "Enabled"
  }
}

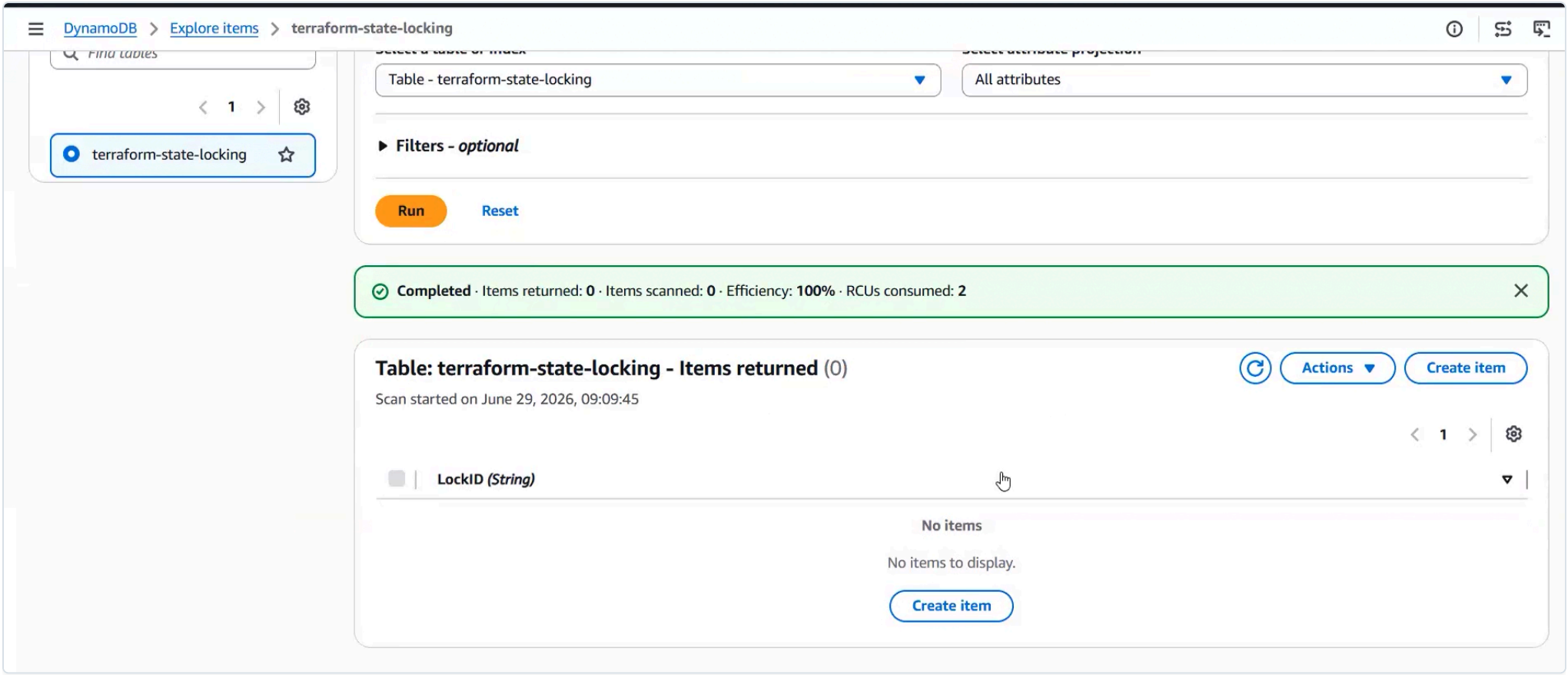
~
~
~
```

```
resource "aws_dynamodb_table" "example_table" {
  name           = "terraform-state-locking"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "LockID"  ### primary key must be “LockID”

  attribute {
    name = "LockID"
    type = "S" # S for String
  }

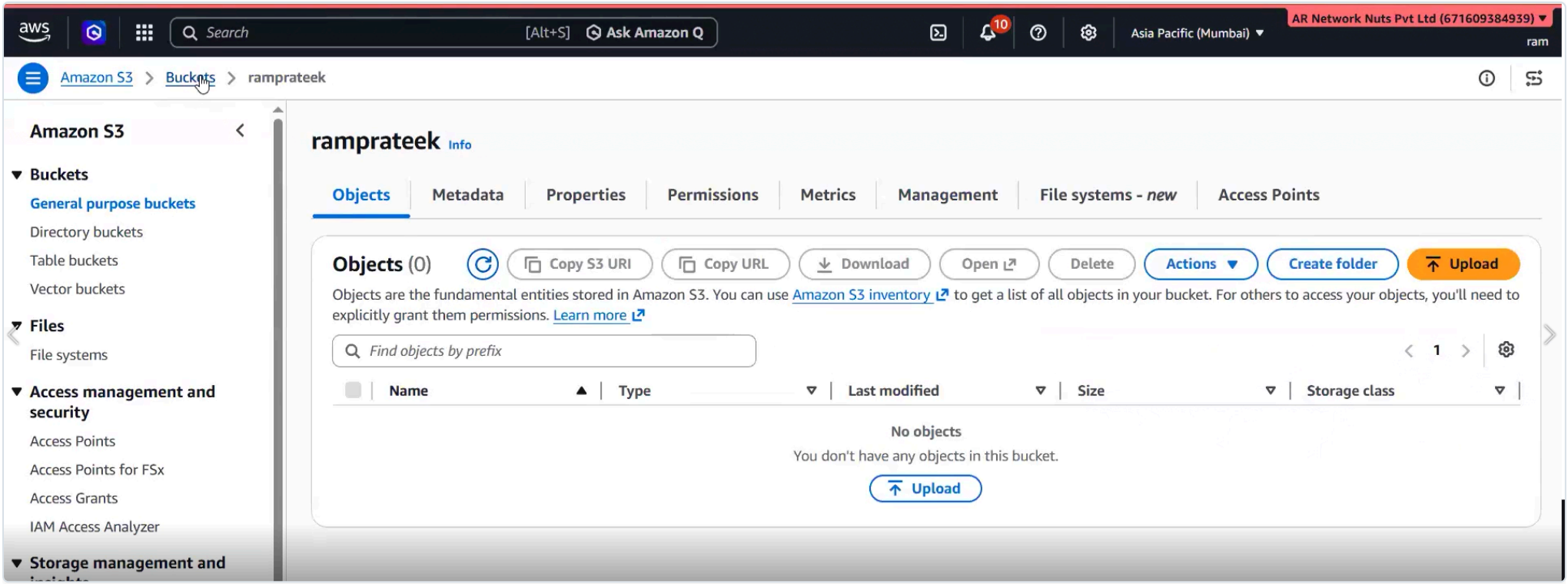
  tags = {
    Environment = "Development"
    Project     = "Example"
  }
}
```


[root@Devops-server project1]# terraform state list
aws_dynamodb_table.example_table
aws_s3_bucket.example
aws_s3_bucket_versioning.versioning_example
[root@Devops-server project1]#

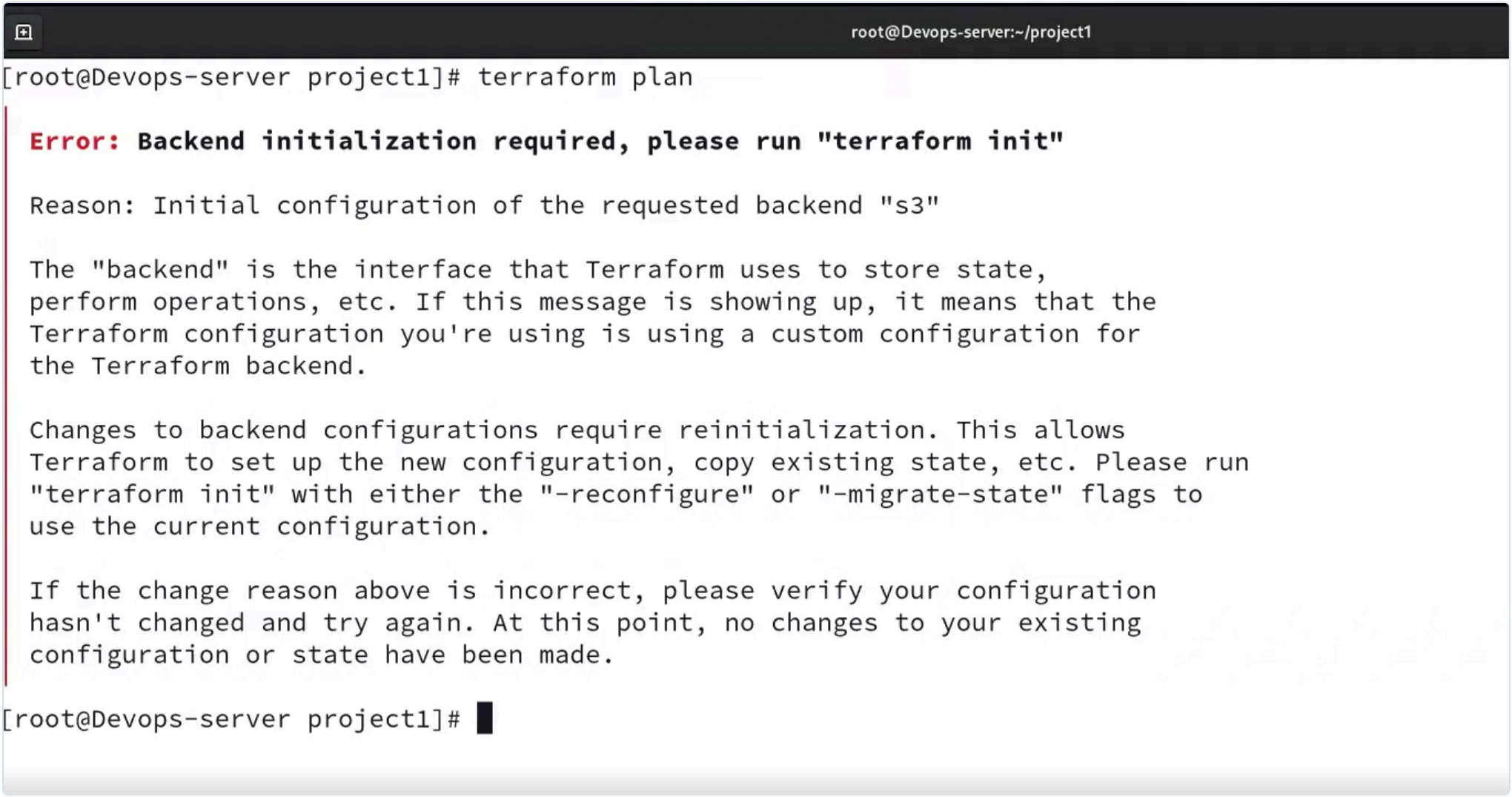

DynamoDB > Explore items > terraform-state-locking
Find tables
1
terraform-state-locking
Table - terraform-state-locking
All attributes
Filters - optional
Run
Reset
Completed · Items returned: 0 · Items scanned: 0 · Efficiency: 100% · RCUs consumed: 2
Table: terraform-state-locking - Items returned (0)
Scan started on June 29, 2026, 09:09:45
Actions
Create item
1
LockID (String)
No items
No items to display.
Create item

```
[root@Devops-server project1]# ll
total 16
-rw-r--r--. 1 root root  643 Jun 29 09:08 main.tf
-rw-r--r--. 1 root root 6540 Jun 29 09:09 terraform.tfstate
-rw-r--r--. 1 root root 3970 Jun 29 09:09 terraform.tfstate.backup
[root@Devops-server project1]#
```

S3 bucket rampratеek — 0 objects




root@Devops-server:~/project1 — vim back.tf
terraform {
backend "s3" {
bucket = "ramprateek"
key = "state/terraform.tfstate"
dynamodb_table = "terraform-state-locking"
}
}


root@Devops-server:~/project1
[root@Devops-server project1]# terraform plan
Error: Backend initialization required, please run "terraform init"
Reason: Initial configuration of the requested backend "s3"
The "backend" is the interface that Terraform uses to store state,
perform operations, etc. If this message is showing up, it means that the
Terraform configuration you're using is using a custom configuration for
the Terraform backend.
Changes to backend configurations require reinitialization. This allows
Terraform to set up the new configuration, copy existing state, etc. Please run
"terraform init" with either the "-reconfigure" or "-migrate-state" flags to
use the current configuration.
If the change reason above is incorrect, please verify your configuration
hasn't changed and try again. At this point, no changes to your existing
configuration or state have been made.
[root@Devops-server project1]#

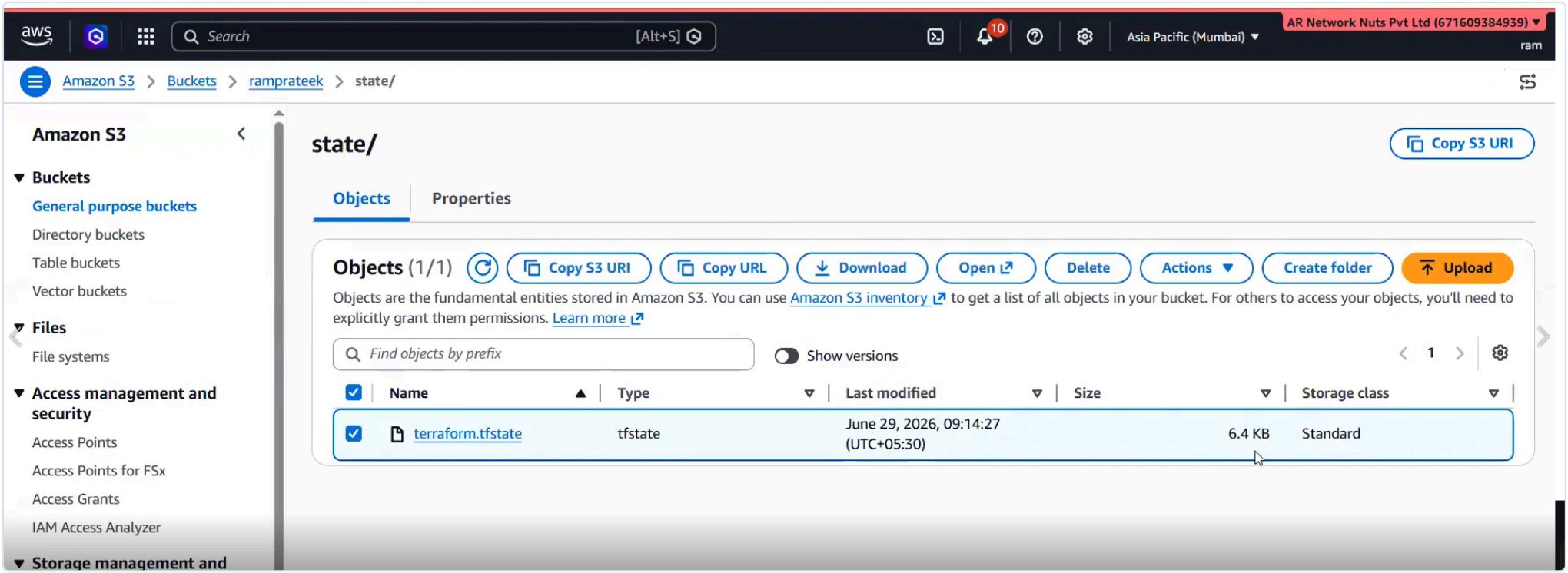
```
[root@Devops-server project1]# terraform init
Initializing the backend...
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "local" backend to the
  newly configured "s3" backend. No existing state was found in the newly
  configured "s3" backend. Do you want to copy this state to the new "s3"
  backend? Enter "yes" to copy and "no" to start with an empty state.

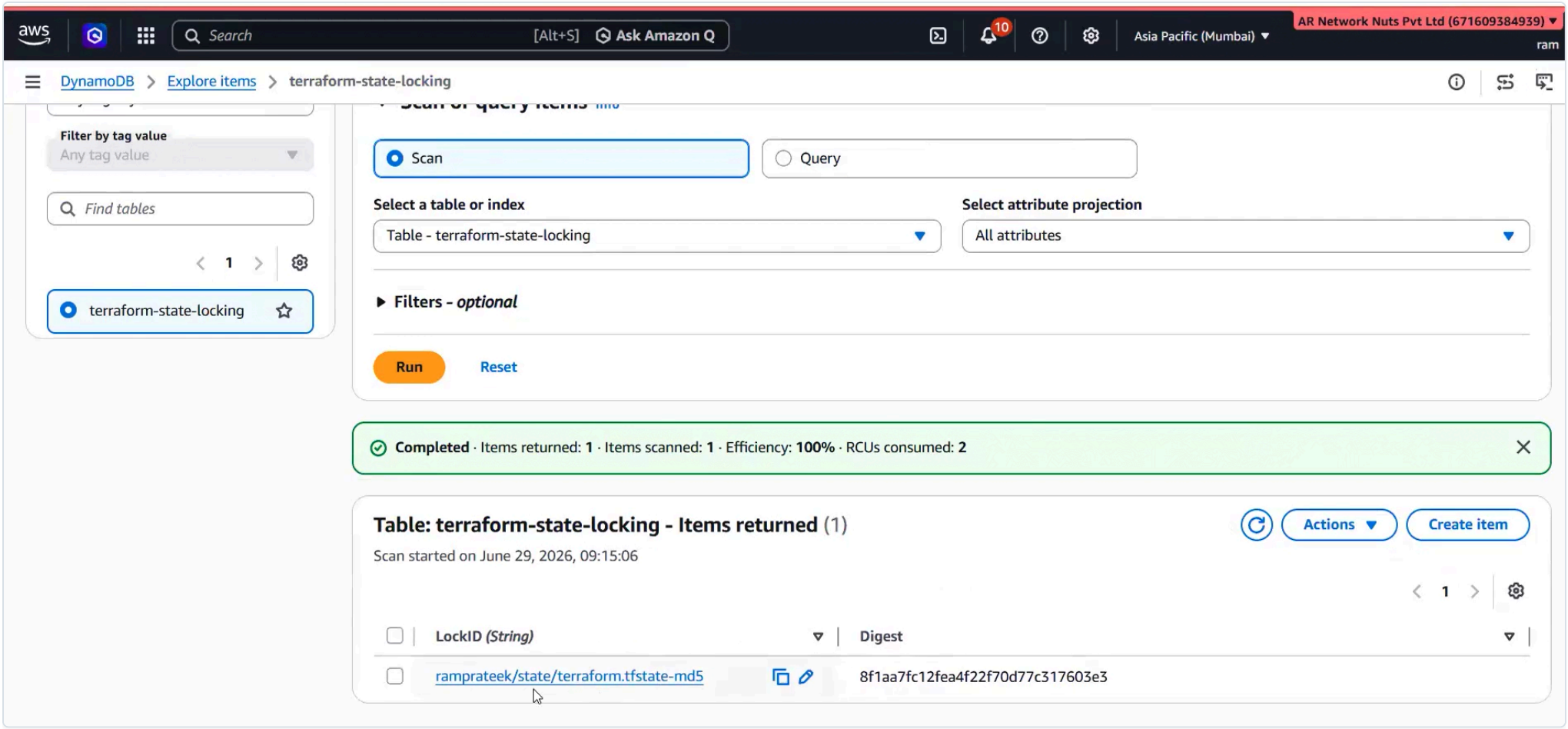
  Enter a value:
```

```
[root@Devops-server project1]# terraform init
Initializing the backend...
Do you want to copy existing state to the new backend?
  Pre-existing state was found while migrating the previous "local" backend to the
  newly configured "s3" backend. No existing state was found in the newly
  configured "s3" backend. Do you want to copy this state to the new "s3"
  backend? Enter "yes" to copy and "no" to start with an empty state.

  Enter a value: yes


Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
```


aws
Search
[Alt+S]
10
Asia Pacific (Mumbai)
AR Network Nuts Pvt Ltd (671609384939)
ram
Amazon S3 > Buckets > rampateek > state/
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and
state/
Copy S3 URI
Objects
Properties
Objects (1/1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Show versions
1
Name
Type
Last modified
Size
Storage class
terraform.tfstate
tfstate
June 29, 2026, 09:14:27 (UTC+05:30)
6.4 KB
Standard

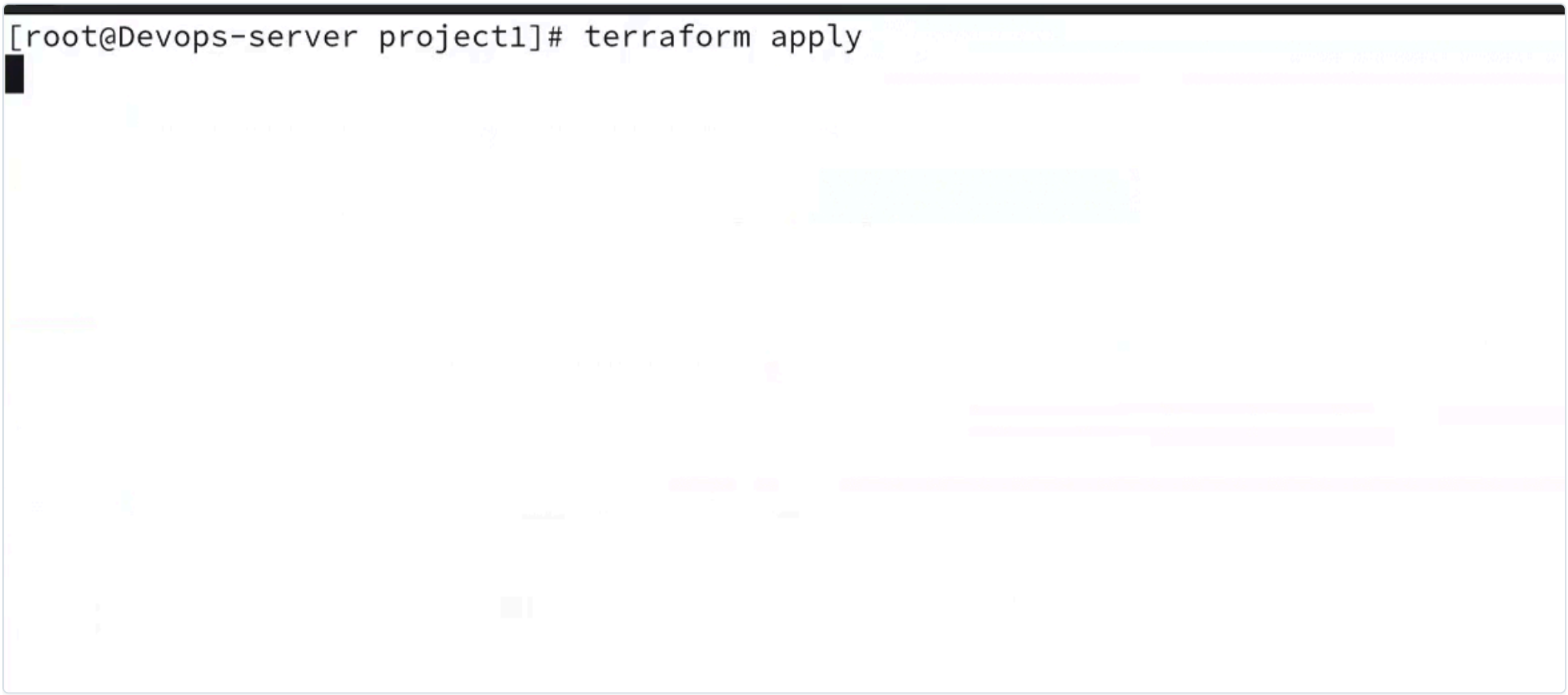

aws
Search
[Alt+S]
Ask Amazon Q
10
Asia Pacific (Mumbai)
AR Network Nuts Pvt Ltd (671609384939)
ram
DynamoDB
Explore items
terraform-state-locking
Filter by tag value
Any tag value
Find tables
1
terraform-state-locking
Scan
Query
Select a table or index
Table - terraform-state-locking
Select attribute projection
All attributes
Filters - optional
Run
Reset
Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2
Table: terraform-state-locking - Items returned (1)
Scan started on June 29, 2026, 09:15:06
Actions
Create item
1
LockID (String)
Digest
ramprateek/state/terraform.tfstate-md5
8f1aa7fc12fea4f22f70d77c317603e3

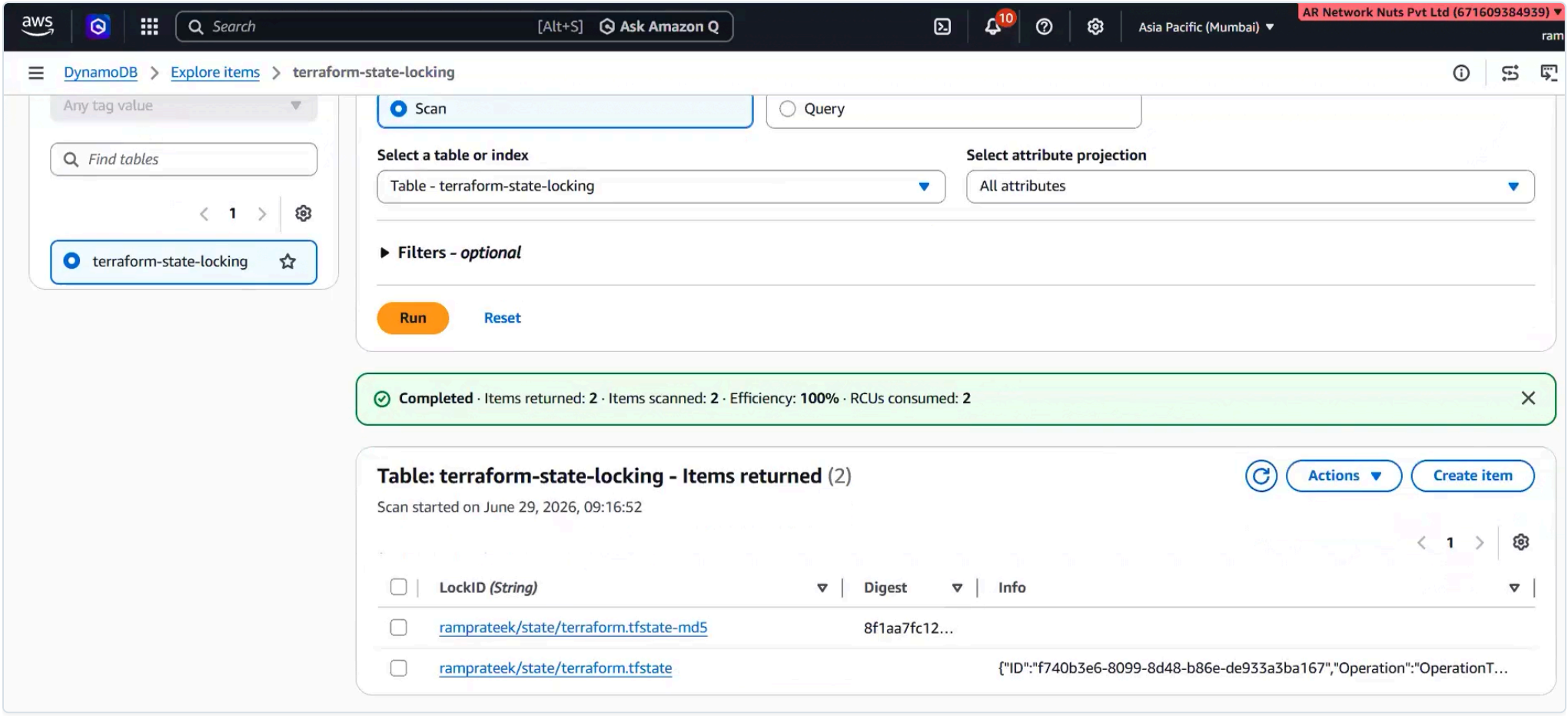
root@Devops-server:~/project1 — vim ec2.tf

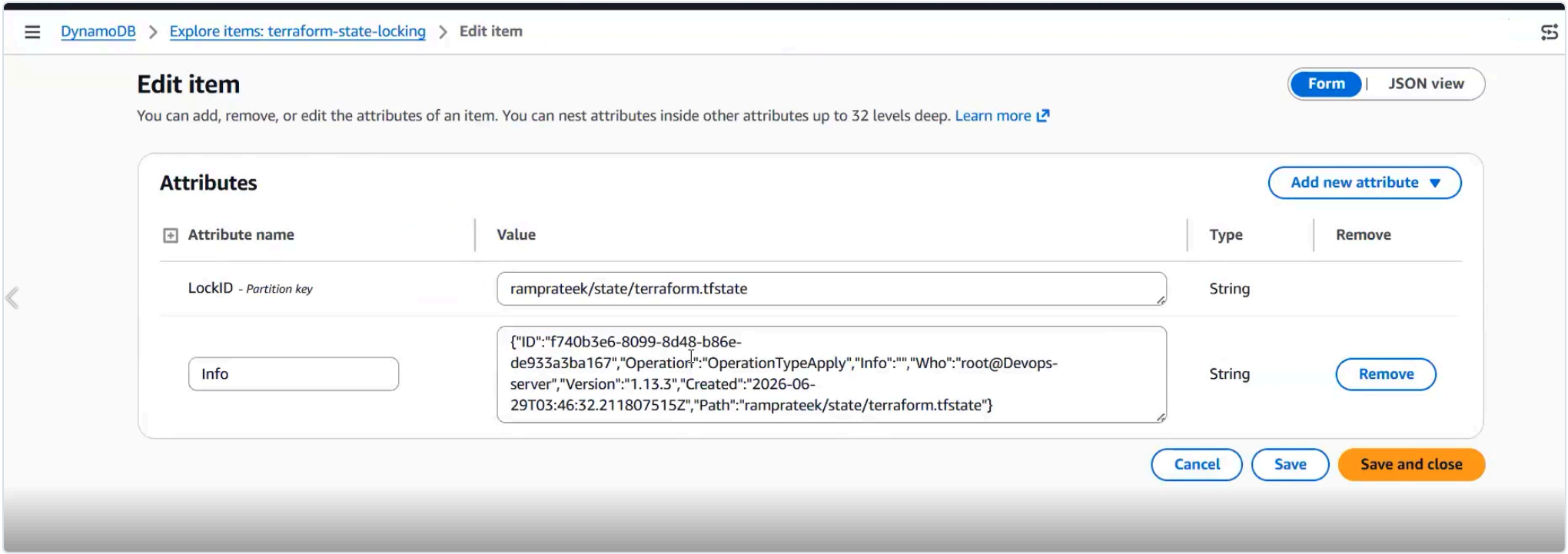
```
resource "aws_instance" "linux" {
  ami           = "ami-0cca134ec43cf708f"
  instance_type = "t2.micro"
  key_name      = "ram_aws"
  tags = {
    Name = "Test_server"
    source = "terraform"
  }

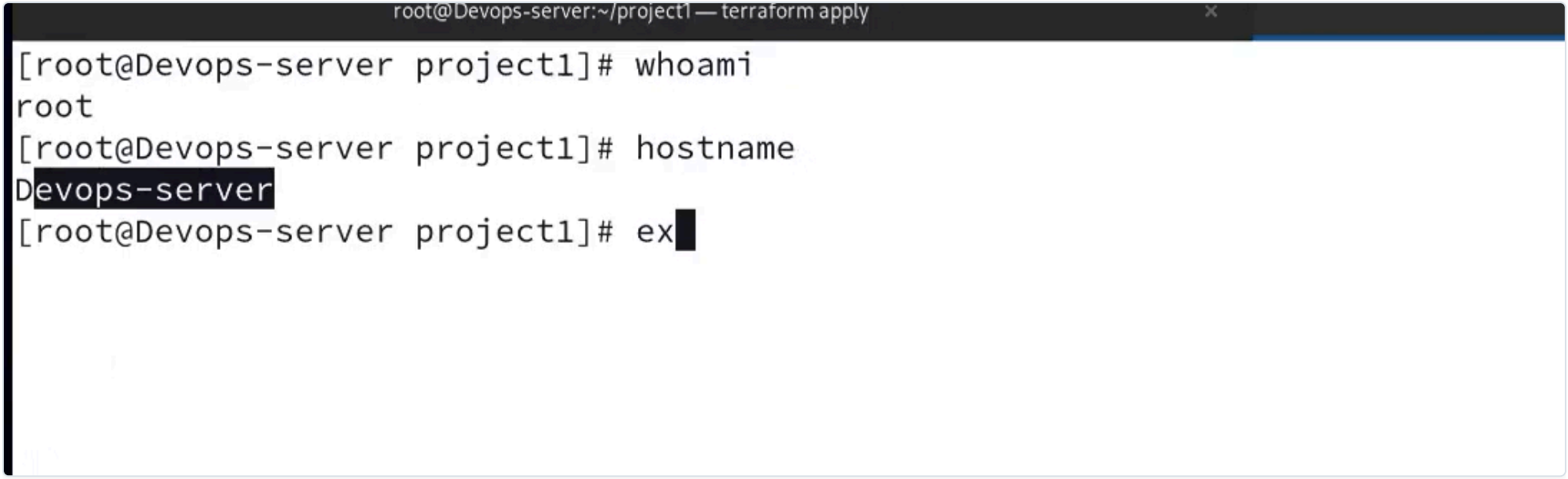
}
```

`terraform apply` started (held at the approval prompt)




aws
Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
AR Network Nuts Pvt Ltd (671609384939)
ram
DynamoDB
Explore items
terraform-state-locking
Any tag value
Find tables
terraform-state-locking
Scan
Query
Select a table or index
Table - terraform-state-locking
Select attribute projection
All attributes
Filters - optional
Run
Reset
Completed · Items returned: 2 · Items scanned: 2 · Efficiency: 100% · RCUs consumed: 2
Table: terraform-state-locking - Items returned (2)
Scan started on June 29, 2026, 09:16:52
Actions
Create item
LockID (String)
Digest
Info
ramprateek/state/terraform.tfstate-md5
8f1aa7fc12...
ramprateek/state/terraform.tfstate
{"ID":"f740b3e6-8099-8d48-b86e-de933a3ba167","Operation":"OperationT...


DynamoDB
Explore items: terraform-state-locking
Edit item
Edit item
You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. Learn more
Form
JSON view
Attributes
Add new attribute
Attribute name
Value
Type
Remove
LockID - Partition key
ramprateek/state/terraform.tfstate
String
Info
{"ID":"f740b3e6-8099-8d48-b86e-de933a3ba167","Operation":"OperationTypeApply","Info":"","Who":"root@Devops-server","Version":"1.13.3","Created":"2026-06-29T03:46:32.211807515Z","Path":"ramprateek/state/terraform.tfstate"}
String
Remove
Cancel
Save
Save and close


root@Devops-server:~/project1 — terraform apply
[root@Devops-server project1]# whoami
root
[root@Devops-server project1]# hostname
Devops-server
[root@Devops-server project1]# ex

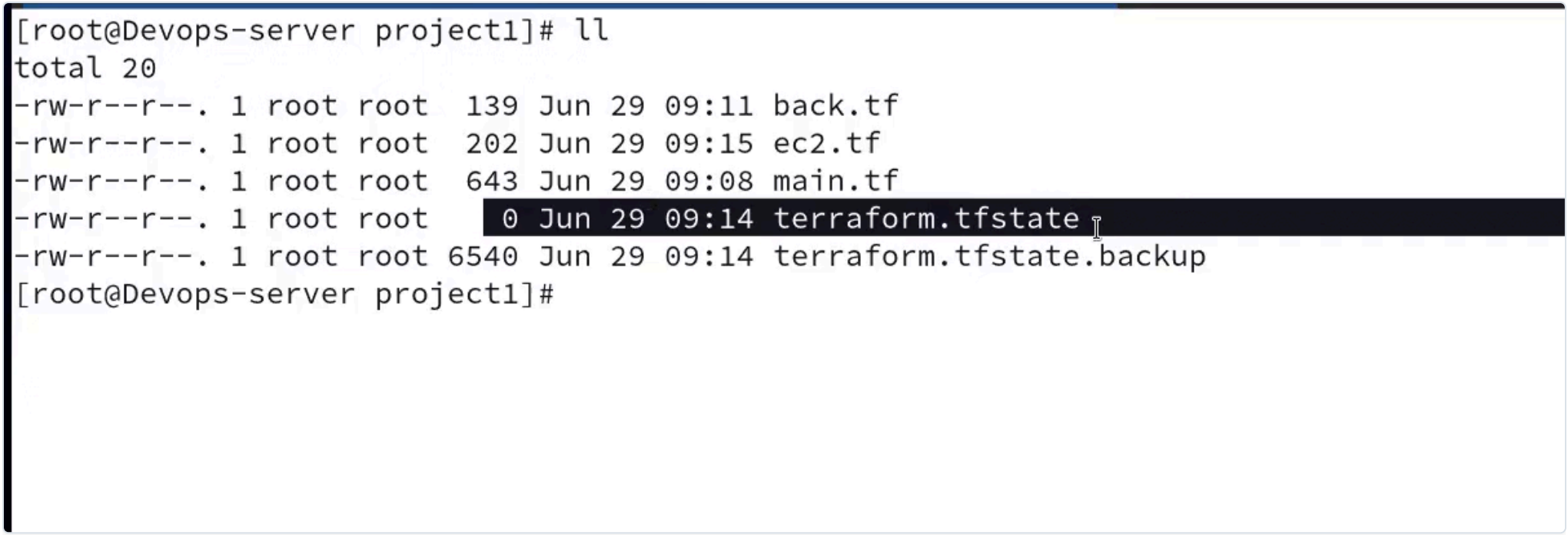
```
[root@Devops-server project1]# terraform plan

Error: Error acquiring the state lock

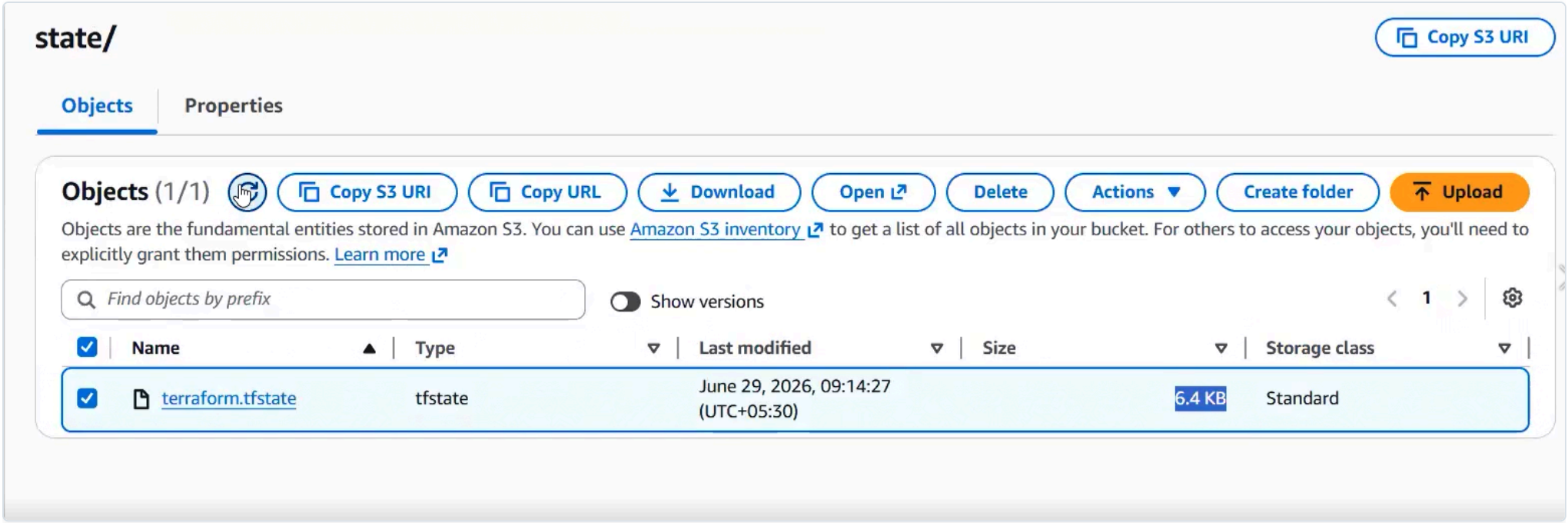
Error message: operation error DynamoDB: PutItem, https response error StatusCode: 400, RequestID:
8J05A4DDPEHILD27RHAK6HAJDNVV4KQNSO5AEMVJF66Q9ASUAAJG, ConditionalCheckFailedException: The conditional request failed
Lock Info:
  ID:        f740b3e6-8099-8d48-b86e-de933a3ba167
  Path:      ramprateek/state/terraform.tfstate
  Operation: OperationTypeApply
  Who:       root@Devops-server
  Version:   1.13.3
  Created:   2026-06-29 03:46:32.211807515 +0000 UTC
  Info:


Terraform acquires a state lock to protect the state from being written
by multiple users at the same time. Please resolve the issue above and try
again. For most commands, you can disable locking with the "-lock=false"
flag, but this is not recommended.

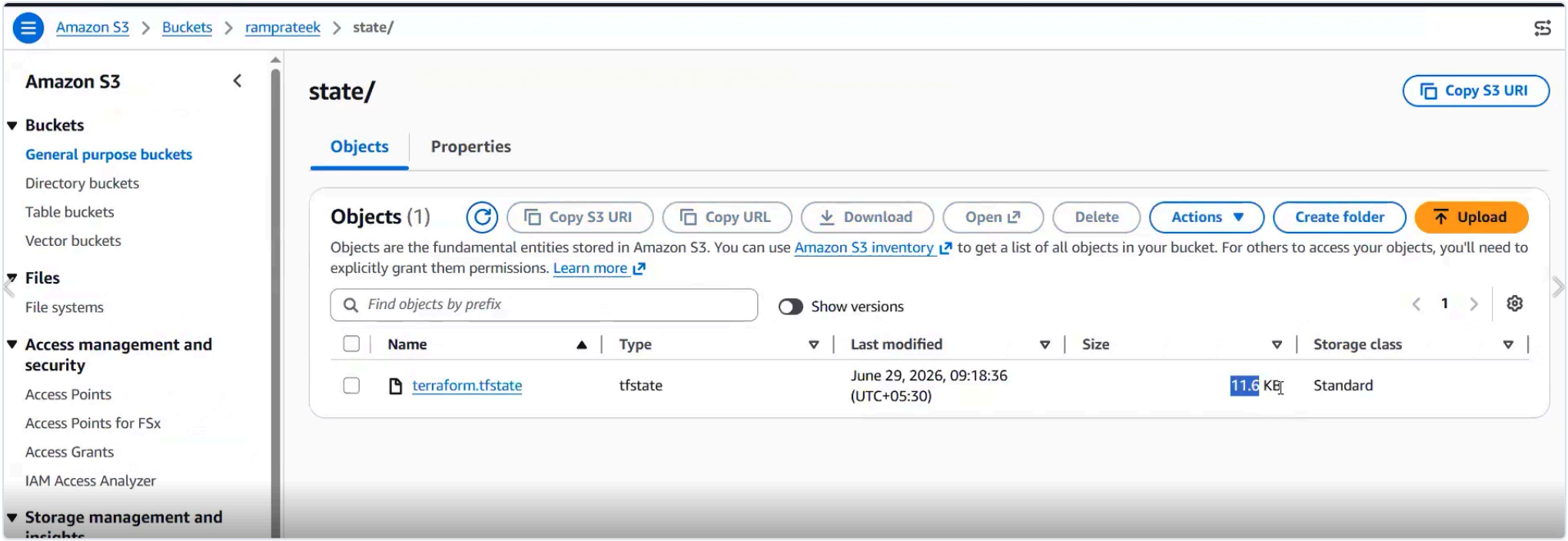
[root@Devops-server project1]#
```


[root@Devops-server project1]# ll
total 20
-rw-r--r--. 1 root root 139 Jun 29 09:11 back.tf
-rw-r--r--. 1 root root 202 Jun 29 09:15 ec2.tf
-rw-r--r--. 1 root root 643 Jun 29 09:08 main.tf
-rw-r--r--. 1 root root   0 Jun 29 09:14 terraform.tfstate
-rw-r--r--. 1 root root 6540 Jun 29 09:14 terraform.tfstate.backup
[root@Devops-server project1]#

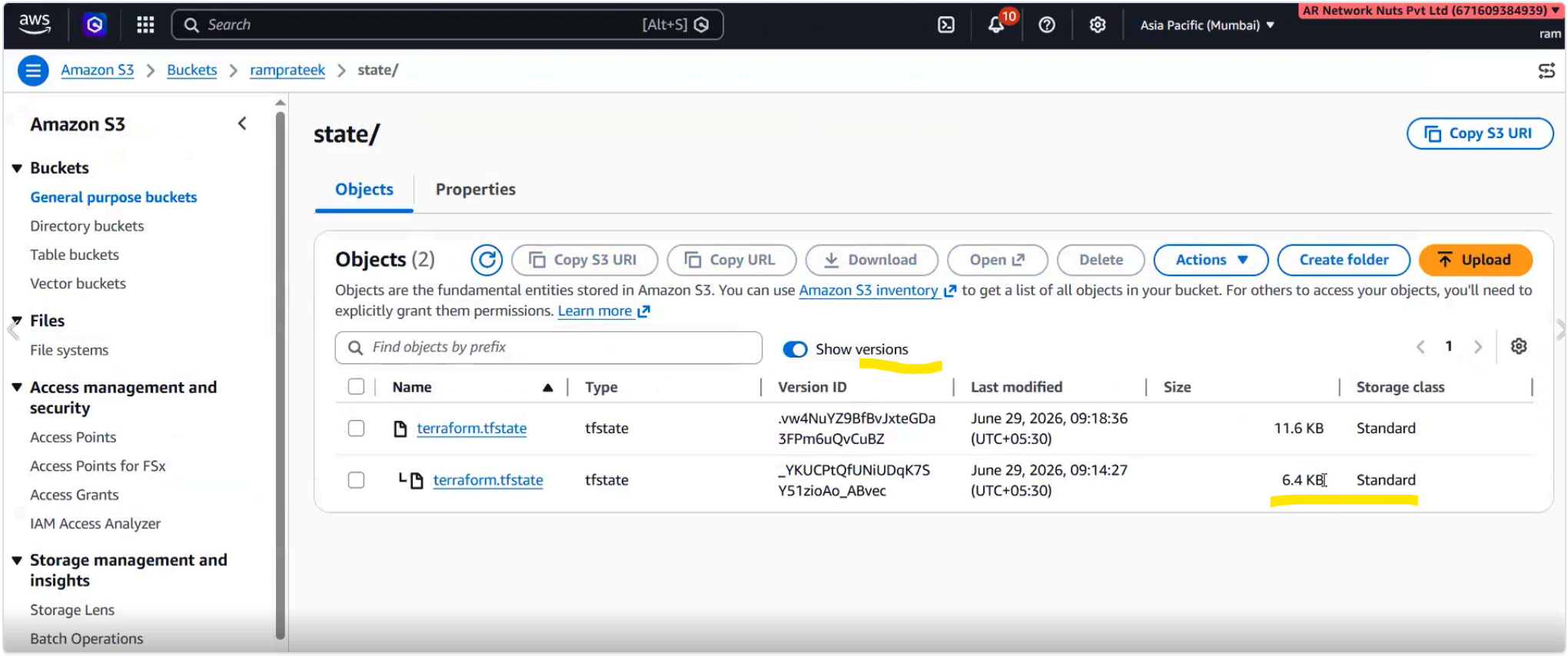
S3 state still 6.4 KB before the EC2 apply finished



After apply — S3 state grows to 11.6 KB (09:18:36)



Show versions — current 11.6 KB + previous 6.4 KB version kept (benefit of versioning)



Back to local — `rm back.tf`


```
[root@Devops-server project1]# rm back.tf
rm: remove regular file 'back.tf'? y
[root@Devops-server project1]#
```

```
[root@Devops-server project1]# rm back.tf
rm: remove regular file 'back.tf'? y
[root@Devops-server project1]# terraform plan

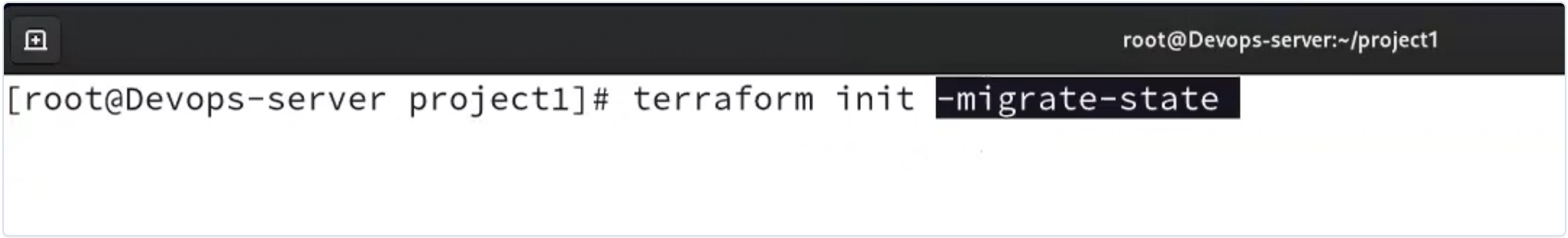
  Error: Backend initialization required, please run "terraform init"

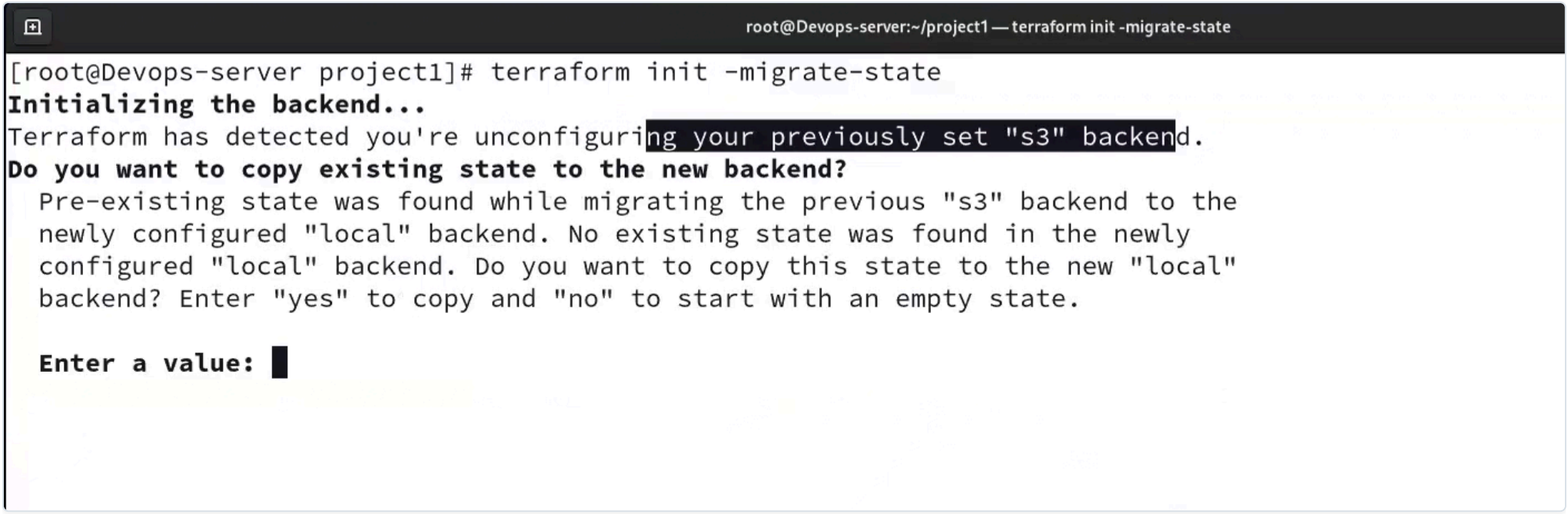
  Reason: Unsetting the previously set backend "s3"

  The "backend" is the interface that Terraform uses to store state,
  perform operations, etc. If this message is showing up, it means that the
  Terraform configuration you're using is using a custom configuration for
  the Terraform backend.

  Changes to backend configurations require reinitialization. This allows
  Terraform to set up the new configuration, copy existing state, etc. Please run
  "terraform init" with either the "-reconfigure" or "-migrate-state" flags to
  use the current configuration.

  If the change reason above is incorrect, please verify your configuration
  hasn't changed and try again. At this point, no changes to your existing
  configuration or state have been made.
```


root@Devops-server:~/project1
[root@Devops-server project1]# terraform init -migrate-state


root@Devops-server:~/project1 — terraform init -migrate-state
[root@Devops-server project1]# terraform init -migrate-state
Initializing the backend...
Terraform has detected you're unconfiguring your previously set "s3" backend.
Do you want to copy existing state to the new backend?
Pre-existing state was found while migrating the previous "s3" backend to the
newly configured "local" backend. No existing state was found in the newly
configured "local" backend. Do you want to copy this state to the new "local"
backend? Enter "yes" to copy and "no" to start with an empty state.
Enter a value: